[negated]). Or maybe we want to set scores based on custom requirements rather than using Elasticsearch's built-in relevance algorithms.

The individual leaf queries we've worked with so far are limited in that they can search based on one or several criteria but not based on more complex requirements (for example, searching for books written by a specific author, published between certain dates, listed as a best-seller or rated 4.5 out of 5, with a specific number of pages). Such advanced queries require advanced searching query capabilities. We look at these compound queries in this chapters.

*Compound queries* are advanced search constructs to query complex search criteria in Elasticsearch. They are made of individual leaf queries wrapped in conditional clauses and other constructs to provide capabilities such as letting the user set custom scores using predefined functions, boosting positive search matches while suppressing negative clauses, developing scores using scripts, and so on. They allow us to use individual leaf queries to develop fully fledged advanced queries of various types.

In this chapter, we examine the requirements that compound queries satisfy, and their semantics and usage. We look at Boolean queries, where multiple leaf queries are cast in a few conditional clauses to design an advanced search query, and we use clauses like `must`, `must_not`, `should`, and `filter` to arrange the leaf queries into a compound query. We look at `boosting` queries to enhance the query's score when a positive match occurs while simultaneously lowering the score for negations. We then learn about a predefined static scoring query called `constant_score`, which helps set a static score on the returned results.

We also run through function score queries to help set user-defined custom scoring algorithms using a set of functions. We look at the mechanics of setting scores using scripts and weights based on other field values present in the document, as well as random numbers. But first, let's get some sample data.

## 11.1 Sample product data

In this chapter's examples, we work with a data set of electrical and electronic products such as televisions (TVs), laptops, mobile phones, refrigerators (fridges), and so on. The product data is available on GitHub (http://mng.bz/Rxwa) and on the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition). In this section, we review the definition and indexing processes.

### 11.1.1 The products schema

The first step in developing the `products` index is to create a data schema defining the fields and their data types. The following listing shows the schema at a glance (the full schema is available with the book's files).

**Listing 11.1 Defining the `products` schema**

```
PUT products
{
  "mappings": {
```